

UAV Path Visualizer: Capstone Project Report

Lethabo Neo
Department of Computer Science
nxxlet001@myuct.ac.za

Max Mkhabela
Department of Information Systems
mkhmax003@myuct.ac.za

Kenneth Baloyi
Department of Computer Science
blyken007@myuct.ac.za

1. Approach

Abstract

This project develops a 3D visualization system for Unmanned Aerial Vehicle (UAV) flight trajectories, enabling users to preview, compare, and analyze paths generated by flight planning algorithms. The system supports loading 3D models of structures, trajectory data in JSON format, rendering smooth paths with spline interpolation, visualizing waypoints with orientation markers, and providing a first-person view (FPV) preview. It includes multi-trajectory comparisons with metrics such as path length, vertical displacement, cumulative turning angle, and estimated duration. The system allows real-time waypoint editing with FPV updates and optimized rendering for large trajectories. It provides an interactive GUI for UAV operators to evaluate and refine flight paths for photogrammetric surveying of cultural heritage sites. The system is compatible with Windows and Linux platforms, with known limitations on macOS due to threading restrictions in the GUI framework.

1.1. Introduction

Unmanned Aerial Vehicles (UAVs or drones) are increasingly used for photogrammetric reconstruction of terrestrial structures, such as cultural heritage sites, where high-quality images are captured to build detailed 3D models. Manual flight path planning is time-consuming and inefficient, leading to research in automated algorithms that generate optimal paths minimizing flight time and image count. This project focuses on creating a 3D visualization system for UAV flight trajectories, allowing users to preview paths, compare trajectories from different algorithms, and export mission data for real-world deployment.

The system addresses key challenges in UAV path planning by providing interactive tools for loading 3D scenes (e.g., OBJ/PLY models), parsing waypoint data (positions, yaw, pitch), interpolating smooth curves, and calculating efficiency metrics. Users can edit waypoints in real-time, view FPV simulations, and compare multiple trajectories side-by-side. The coordinate system uses XYZ for positions (meters) and yaw/pitch for orientations (degrees), with roll assumed level for stability.

This project introduces an educational tool designed to visualize and measure optimal UAV flight paths in an engaging and interactive manner. It leverages Python with Open3D and Trimesh to create a rich experience where operators load models, visualize trajectories, and analyze metrics. The core lies in its emphasis on usability, with features like FPV previews and waypoint editing.

The system caters to learners and professionals, from beginners to researchers in geospatial analysis. Users can comfortably navigate and utilize the tool for path optimization.

1.1.1. Software Engineering Methods

The project adopted an Agile methodology with iterative development cycles. This approach enabled weekly sprints and risk-driven phases including inception for scope definition and risk assessment, elaboration for modeling and architecture, construction for implementation, and transition for testing and

deployment. The iterative approach proved beneficial for handling the complexity of 3D rendering and threading requirements.

1.2. Requirements Captured

This section covers functional, non-functional, and usability requirements, derived from Stage 1 Scope and Stage 2 modeling.

Functional Requirements:

- Loading and rendering 3D models in OBJ/PLY formats
- Parsing trajectory data from JSON files with waypoint positions and orientations
- Rendering smooth flight paths using linear and spline interpolation
- Providing first-person view (FPV) preview from waypoint perspectives
- Interactive waypoint editing with real-time updates
- Multi-trajectory comparison with efficiency metrics
- Video recording of flight simulations
- Export functionality for modified trajectories

Non-functional Requirements:

- Performance: Real-time rendering at 20-30 FPS for trajectories up to 1000 waypoints
- Usability: Intuitive GUI with collapsible panels and keyboard shortcuts
- Portability: Cross-platform compatibility (Windows/Linux, with macOS limitations)
- Reliability: Graceful error handling for invalid files and edge cases
- Maintainability: Modular architecture with clear separation of concerns

Use Case Narratives:

Use Case 1: First-Time Scene Loading and Exploration

Actor: UAV Operator

Description: UAV operator loads a 3D model into the application to display the scene for path planning.

Trigger: The operator needs to view a 3D structure before analyzing the flight path.

Preconditions: The UAV path visualizer application is running.

Postconditions: 3D model is rendered in the main viewport.

Flow: A UAV operator has just received a 3D model of some cultural heritage site that needs surveying. They perform an action that allows them to load a 3D model and select the valid file. The system loads the model, from a default bird's eye view, and displays the 3D structure on a reference ground plane, with axes shown in the corner for orientation. The operator can manipulate the camera scene via defined inputs.

Alternative Paths:

- *Invalid File:* The operator selects an unsupported file. The system rejects and displays an error message and displays the supported file types. The use case can continue.
- *Corrupt Model file:* The system is unable to parse the file. The system does not crash but will display an error message indicating the file is corrupt and cannot be loaded. The use case terminates.

- *Missing material file:* The system successfully parses the model file but cannot locate associated material or texture image files referenced within it. The system will render the 3D model using default, untextured material. The system displays a non-blocking warning message and use case continues.

Use Case 2: Visualizing and Understanding a Flight Path

Actor: UAV Operator

Description: The operator inspects a specific point on the flight path to see the drone's camera perspective.

Trigger: The operator needs to evaluate a flight path.

Preconditions: The UAV path visualizer application is running.

Postconditions: Flight path is rendered in the scene as a navigable 3D curve with orientation markers.

Flow: The operator wants to see the proposed flight path. They initiate action to load a trajectory file. Operator will select valid trajectory file. The system will process the file and output flight path. Objects are shown along the line showing where the drone took a picture and the orientation of camera will be pointing.

Alternative Paths:

- *Corrupt File:* The system attempts to parse the file but encounters a structural error or missing data. The system will display a specific error message that will detail where the error or missing data is. The use case terminates.
- *Trajectory Scene mismatch:* The system successfully parses the trajectory file but detects metadata in file does not match with currently loaded 3D model. The system displays a warning message and use case terminates.
- *Path is outside visible bounds:* The system renders the path, but its coordinates are outside the current camera's view. The system detects that the newly added object is not visible and asks the user to automatically adjust to fit the path. The operator confirms, the system automatically adjusts camera's position and zoom to frame both 3D model and trajectory, use case continues.

Use Case 3: In-Depth Waypoint Inspection with First-Person View (FPV)

Actor: UAV Operator

Trigger: The operator needs to verify the camera's line of sight at a specific waypoint.

Preconditions: A trajectory has been successfully loaded.

Postconditions: FPV for a selected waypoint is displayed.

Flow: The operator selects a waypoint marker either in 3D scene or from waypoint list panel. It now shows exactly what the drone's camera would see from that specific point.

Alternative Paths:

- *No Waypoint selected:* The FPV window displays a default state. Use case continues.
- *User deselects waypoint:* The system removes the highlighting from waypoint marker. The FPV window reverts to its default state. Use case continues.

Other Analysis Artifacts:

We produced an overall architecture diagram (see Section 1.3) and class hierarchy showing the relationships between core components including Scene3D, TrajectoryParser, RenderManager, and the main application controller.

1.3. Design Overview

The system architecture follows a modular design pattern with clear separation between rendering, data processing, and user interface components. The design emphasizes maintainability and extensibility while addressing performance requirements for real-time 3D rendering.

System Architecture

The architecture consists of four primary layers:

1. **Presentation Layer:** Tkinter-based GUI with collapsible frames, canvas widgets, and control panels
2. **Application Layer:** Main application controller managing user interactions and coordinating between components
3. **Processing Layer:** Data processing modules for trajectory parsing, interpolation, and metric calculations
4. **Rendering Layer:** Open3D-based 3D rendering with multi-threaded architecture for performance optimization

Key Classes/Components

- **UAVVisualizationApp:** Main application controller managing GUI and coordinating all subsystems
- **Scene3D:** Handles 3D model loading using Trimesh and Open3D, manages scene geometry
- **TrajectoryParser:** Processes JSON trajectory files, supports multiple waypoint formats
- **RenderManager:** Manages multiple rendering threads, handles frame queues and synchronization
- **Open3DRenderThread:** Dedicated rendering thread for non-blocking 3D visualization
- **TrajectoryInterpolator:** Implements linear and spline interpolation algorithms
- **VideoRecorder:** Handles video export using OpenCV for flight simulations

Interface Architecture

The GUI employs a three-panel layout with collapsible sections:

- **Left panel:** File operations, camera controls, trajectory settings, waypoint editing
- **Center panel:** Main 3D viewport with mouse and keyboard interaction
- **Right panel:** Trajectory information, first-person view, comparison results

Rationale for Design Choices

- **Multi-threading:** Separate rendering threads prevent GUI blocking during intensive 3D operations
- **Modular architecture:** Clear component separation enables independent testing and future extensions
- **Queue-based communication:** Bounded queues prevent memory overflow and ensure responsive interaction
- **Dual rendering pipeline:** Separate main and FPV renderers allow simultaneous viewpoints

Special Techniques

- **Adaptive interpolation:** Automatic fallback from spline to linear interpolation for edge cases
- **Performance optimization:** Object pooling for waypoint markers, batched updates during editing
- **Cross-platform compatibility:** Platform-specific handling for threading limitations
- **Memory management:** Frame queue limits and cleanup procedures prevent memory leaks

1.4. Implementation

Data Structures

The core data structures are designed for efficiency and clarity:

- **Waypoint**: Contains 3D position (NumPy array), yaw/pitch angles (float), and index
- **Trajectory**: Manages waypoint collections with metadata including name, color, and interpolation method
- **Camera3D**: Represents viewpoint with position, orientation, and field of view parameters

User Interface Implementation

The GUI is implemented using Tkinter with custom widgets:

- **CollapsibleFrame**: Custom widget allowing sections to be expanded/collapsed
- **Mouse interaction**: Drag for camera rotation, wheel for zoom, click for focus
- **Keyboard controls**: WASD for movement, arrows for rotation, space for waypoint creation
- **Real-time feedback**: Progress labels and status updates during operations

Significant Methods

Key algorithms implemented include:

- `TrajectoryInterpolator.interpolate_spline()`: Cubic spline interpolation using SciPy with robust error handling
- `RenderManager.render_loop()`: Asynchronous rendering with frame rate limiting and performance monitoring
- `generate_camera_visualization()`: Creates 3D wireframe camera representations with proper orientation
- `calculate_detailed_metrics()`: Computes path efficiency metrics including turning angles and vertical displacement

Special Programming Techniques and Libraries

- **Open3D**: Primary 3D rendering library for geometry processing and visualization
- **Trimesh**: Mesh processing for robust model loading and manipulation
- **Threading**: Producer-consumer pattern with bounded queues for render pipeline
- **NumPy**: Efficient array operations for geometric calculations
- **SciPy**: Advanced interpolation algorithms with automatic method selection
- **OpenCV**: Video recording and frame processing capabilities

Table 1: Summary Testing Plan.

Process	Technique
1. Unit Testing: Individual class and method validation	White-box testing with edge cases, mock data validation
2. Integration Testing: Component interaction verification	Behavioral testing with real trajectory data, renderer synchronization tests
3. Validation Testing: User requirement satisfaction	Use-case based black box testing, acceptance criteria verification
4. System Testing: Platform and performance evaluation	Cross-platform compatibility, stress testing with large datasets

Table 2: Test Cases and Results.

Test Case	Input	Expected Result	Actual Result
Application Launch	Execute vis.py	GUI loads successfully	Pass: GUI loads on Windows/Linux
Load 3D Model	Valid .obj file	Model renders with textures	Pass: Model displays correctly
Load Trajectory	JSON waypoint file	Trajectory path visible	Pass: Spline path with markers
Platform Compatibility	Run on macOS	Application functions	Fail: Threading exception occurs
Large Dataset	1000+ waypoints	Responsive rendering	Pass: Maintains 20+ FPS
Invalid Input	Corrupted JSON	Graceful error handling	Pass: Error message displayed
Memory Usage	Extended operation	No memory leaks	Pass: Stable memory usage
Video Recording	Flight simulation	MP4 file created	Pass: Video saved successfully

1.5. Program Validation and Verification

The testing strategy employed multiple verification approaches to ensure system reliability and performance across different platforms and usage scenarios.

Testing Framework

The Quality Management Plan included comprehensive testing at multiple levels:

Test Cases and Results

Platform Compatibility Issues

During testing, a critical compatibility issue was discovered on macOS platforms. The error log revealed:

```
NSInternalInconsistencyException:
'nextEventMatchingMask should only be called from the Main Thread!'
```

This exception occurs because macOS enforces stricter threading restrictions for GUI operations compared to Windows and Linux. The Open3D visualization component attempts to create rendering contexts from background threads, which violates macOS's threading model requirements.

Resolution Attempts

Two approaches were tested to address the macOS compatibility:

1. **Thread Consolidation:** Modified the renderer to execute all GUI operations on the main thread.

While this resolved the crash, it resulted in significantly degraded performance with unacceptable UI responsiveness.

2. **Alternative Architecture:** Investigated platform-specific rendering backends, but this would require substantial architectural changes beyond the project scope.

Given the project constraints and the fact that the primary target platforms (Windows/Linux) function correctly, the decision was made to document this limitation rather than compromise the overall system performance.

User Testing Results

User testing sessions with domain experts yielded positive feedback:

- Intuitive interface design with logical control grouping
- Effective FPV preview for flight path validation
- Useful comparison metrics for trajectory optimization
- Responsive real-time editing capabilities
- Clear visual feedback during operations

Minor usability improvements were implemented based on feedback, including enhanced keyboard shortcuts and better progress indicators during file loading.

1.6. Conclusion

The UAV Path Visualizer project successfully achieved its primary objectives of creating a comprehensive 3D visualization system for UAV flight trajectory analysis. The system demonstrates robust functionality across its core requirements including 3D model loading, trajectory visualization, waypoint editing, and multi-path comparison.

Achievement Summary:

- **Functional Completeness:** All specified requirements were implemented and tested successfully
- **Performance Optimization:** Real-time rendering achieved for large trajectory datasets (1000+ waypoints)
- **User Experience:** Intuitive interface with comprehensive editing capabilities and visual feedback
- **Cross-Platform Support:** Full compatibility with Windows and Linux platforms
- **Extensible Architecture:** Modular design enabling future enhancements and feature additions

Technical Accomplishments:

The implementation demonstrates several technical strengths:

- Multi-threaded architecture preventing GUI blocking during intensive operations
- Robust error handling for invalid inputs and edge cases
- Efficient memory management preventing leaks during extended operation
- Advanced interpolation algorithms with automatic method selection
- Comprehensive testing coverage ensuring system reliability

Limitations and Future Work:

The primary limitation identified is macOS incompatibility due to platform-specific threading restrictions. Future development could address this through:

- Implementation of platform-specific rendering backends
- Investigation of alternative GUI frameworks with better cross-platform threading support
- Development of a web-based interface to eliminate platform dependencies

The system provides a solid foundation for UAV path planning research and practical applications, offering valuable tools for both educational and professional use in the field of photogrammetric surveying.

1.7. User Manual

A comprehensive user manual is provided in Appendix A, including installation instructions, feature tutorials, and troubleshooting guidance. The manual is designed for both novice users and experienced UAV operators, with step-by-step procedures and screenshot references.

2. Conclusion

This report documented the complete development lifecycle of the UAV Path Visualizer system. The project successfully delivered a functional, efficient, and user-friendly tool for UAV trajectory analysis while identifying and documenting platform-specific limitations. The modular architecture and comprehensive testing ensure the system's reliability and maintainability for future enhancements.

A. User Manual

System Requirements:

- Python 3.8+ with required packages (Open3D, Trimesh, NumPy, SciPy, OpenCV)
- Windows 10+ or Linux Ubuntu 18.04+ (macOS not supported)
- Minimum 4GB RAM, dedicated graphics card recommended
- 1GB free disk space for trajectory data and recordings

Installation:

1. Install Python dependencies: `pip install open3d trimesh numpy scipy opencv-python pillow tkinter`
2. Download the application files to a local directory
3. Ensure sample data files are in the Resources folder

Getting Started:

1. Launch the application: `python vis.py`
2. Load a 3D model: File → Load 3D Model (supports .obj, .ply files)
3. Load trajectory data: File → Load Trajectory (.json format)
4. Navigate the 3D scene using mouse controls (drag to rotate, wheel to zoom)
5. Select waypoints from the list to view FPV perspective
6. Edit waypoints using keyboard controls (WASD for movement, arrows for rotation)
7. Compare multiple trajectories by loading additional files
8. Record simulations: Set parameters and click "Start Recording & Simulation"

Key Features:

- **Interactive 3D Visualization:** Real-time navigation of flight paths and 3D models
- **First-Person View:** Preview drone camera perspective at each waypoint
- **Trajectory Comparison:** Side-by-side analysis of multiple flight paths
- **Real-Time Editing:** Modify waypoint positions and orientations with immediate visual feedback
- **Simulation Recording:** Export MP4 videos of flight simulations
- **Metric Analysis:** Comprehensive efficiency calculations and comparisons

Troubleshooting:

- **Application won't start:** Verify Python version and package installations
- **Model loading fails:** Check file format compatibility (.obj, .ply supported)
- **Poor rendering performance:** Close other applications, ensure graphics drivers are updated
- **macOS compatibility:** Use Windows or Linux systems; macOS version has known threading issues
- **Memory errors:** Reduce trajectory complexity or restart application

Keyboard Shortcuts:

- **W/A/S/D:** Move selected camera forward/left/backward/right
- **Q/E:** Move camera up/down
- **Arrow Keys:** Rotate camera (yaw/pitch)
- **Space:** Add waypoint at current camera position
- **H:** Display help dialog
- **Z/X:** Adjust FPV zoom
- **F/V:** Adjust FPV look-ahead distance

B. Platform Compatibility Analysis

Threading Architecture Issue on macOS:

The incompatibility with macOS stems from fundamental differences in GUI threading models:

- **Windows/Linux:** Allow GUI operations from background threads with proper synchronization
- **macOS:** Enforces strict main-thread-only policy for all GUI operations

The error trace indicates that Open3D's visualization component violates macOS threading restrictions when creating rendering contexts from worker threads. This results in the `NSInternalInconsistencyException` terminating the application.

Performance Trade-offs:

Attempts to resolve macOS compatibility by consolidating all operations to the main thread resulted in:

- Successful application startup without crashes
- Severely degraded responsiveness (1-2 second delays for interactions)
- Unacceptable user experience for real-time editing operations

Given that Windows and Linux platforms represent the majority of target users and provide optimal performance, the decision was made to maintain the current architecture while documenting this platform limitation.

C. Code Quality and Documentation

The codebase adheres to professional software development standards:

Code Organization:

- PEP8 compliance for Python style guidelines
- Comprehensive docstrings for all classes and methods
- Logical module separation with clear interfaces
- Consistent naming conventions throughout

Documentation Standards:

- Detailed method signatures with type hints
- Extensive inline comments explaining complex algorithms
- Error handling with informative log messages
- User-facing messages provide clear guidance and next steps

Output Quality: All system outputs are designed for clarity and usability:

- Formatted metric displays with appropriate units
- Progress indicators during long-running operations
- Structured error messages with actionable information
- Professional-quality video exports with proper encoding

References

- [1] Zhou, Q.-Y., Park, J., & Koltun, V. (2018). Open3D: A modern library for 3D data processing. *arXiv preprint arXiv:1801.09847*.
- [2] Dawson-Haggerty, M. et al. (2019). trimesh. <https://trimsh.org/>.
- [3] Torres, M., Pelta, D. A., Verdegay, J. L., & Torres, J. C. (2016). Coverage path planning with unmanned aerial vehicles for 3D terrain reconstruction. *Expert Systems with Applications*, 55, 441-451.
- [4] Snavely, N., Seitz, S. M., & Szeliski, R. (2008). Modeling the world from internet photo collections. *International Journal of Computer Vision*, 80(2), 189-210.